Cost-Sensitive Decision Trees for Imbalanced Classification

A tutorial of Imbalanced Classification on Decision Tree.

Feb 26, 2020 • 8 min read

jupyter

View On GitHub launch binder Open in Colab

Overview

This tutorial divided into 4 pars:

- Imbalanced classfication dataset
- Decision Trees for Imbalanced classfication
- Weighted Decision Tree with Scikit-Learn
- Grid Search Weighted Decision Trees

Imbalanced classfication dataset

we use the make_classification()function to define a synthetic imbalanced two-class classfication dataset. We generate `10,000` examples with an approximate 1:100 minority to majority class ratio

```
from collections import Counter
from sklearn.datasets import make_classification
from matplotlib import pyplot as plt
import numpy as np
```

```
X, y = make_classification(n_samples=10000, n_features=2, n_redundant=0, n_clusters_per_class=1, weights=

# examinate the info of X and y
print(f'The type of X is {type(X)}')
print(f'The type of y is {type(y)}')
print('\n')
print(f'The size of X is {X.shape}')
print(f'The size of y is {y.shape}')
```

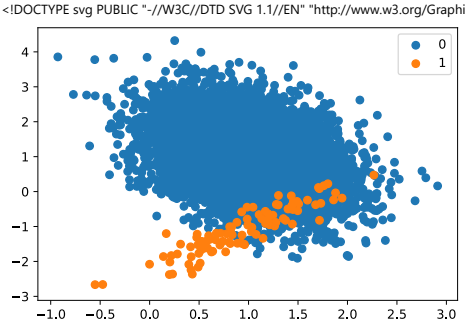
```
The type of X is <class 'numpy.ndarray'>
The type of y is <class 'numpy.ndarray'>


The size of X is (10000, 2)
The size of y is (10000,)
```

```
counter = Counter(y)
print(counter)
```

```
Counter({0: 9900, 1: 100})
```

```
for label, _ in counter.items():
    row_ix = np.where(y == label)[0]
    plt.scatter(X[row_ix, 0], X[row_ix, 1], label=str(label))
plt.legend()
plt.show()
```

<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">



Fit it with standard decision tree model

A decision tree can be defined using the DecisionTreeClassifier class in the scikit-learn library

```
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.tree import DecisionTreeClassifier

# define model
model = DecisionTreeClassifier()
```

```
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)

# evaluate model
score = cross_val_score(model, X, y, scoring='roc_auc', cv=cv, n_jobs=-1)

# summarize performance
print(f'Mean ROC AUC: {np.mean(score):.3f}')
```

```
Mean ROC AUC: 0.734
```

The model performance is achieving a ROC AUC above `0.5` which is `0.734`

This model can be the baseline for comparision for any modification performed to the standard decision tree algorithm

Decision Trees for Imbalanced Classification

The decision tree algorithm is also known as Classification and Regression Trees and involves growing a tree to classify examples from the training dataset.

The tree can be thought to divide the training dataset, where examples progress down the decision points of the tree to arrive in the leaves of the tree and are assigned a class label.

The tree is constructed by splitting the training dataset using values for variables in the dataset. At each point, the split in the data that results in the purest (least mixed) groups of examples is chosen in a greedy manner.

Here, purity means a clean separation of examples into groups where a group of examples of all 0 or all 1 class is the purest, and a 50-50 mixture of both classes is the least pure. Purity is most commonly calculated using Gini impurity, although it can also be calculated using entropy

The calculation of a purity measure involves calculating the probability of an example of a given class being misclassified by a split. Calculating these probabilities involves summing the number of examples in each class within each group.

The splitting criterion can be updated to not only take the purity of the split into account, but also be weighted by the importance of each class.

This can be achieved by replacing the count of examples in each group by a weighted sum, where the coefficient is provided to weight the sum.

Larger weight is assigned to the class with more importance, and a smaller weight is assigned to a class with less importance.

- **Small Weight**: Less importance, lower impact on node purity
- **Large Weight**: More importance, higher impact on node purity

A small weight can be assigned to the majority class, which has the effect of improving (lowering) the purity score of a node that may otherwise look less well sorted. In turn, this may allow more examples from the majority class to be classified for the minority class, better accommodating those examples in the minority class.

As such, this modification of the decision tree algorithm is referred to as a weighted decision tree, a class-weighted decision tree, or a cost-sensitive decision tree.

Modification of the split point calculation is the most common, although there has been a lot of research into a range of other modifications of the decision tree construction algorithm to better accommodate a class imbalance.

Weighted Decision Tree with Scikit-learn

The scikit-learn Python machine learning library provides an implementation of the decision tree algorithm that supports class weighting.

The DecisionTreeClassifier class provides the `class_weight` argument that can be specified as a model hyperparameter. The `class_weight` is a dictionary that defines each class label (e.g. 0 and 1) and the weighting to apply in the calculation of group pruity for splits in the decision tree when fitting the model

For example:

```
# define model
weights = {0:1.0, 1:1.0}
model = DecisionTreeClassifier(class_weight=weights)
```

The class weighting can be defined multiple ways, for example:

- **Domain Expertise**: determined by talking to subject matter experts

- **Tuning**: determined by hyperparameter search such as GirdSearch
- **Huristic**: spcified using a general best practice

A best practice for using the class weighting is to use the inverse of the class distribution present in the training dataset.

For example, the class distribution of the test dataset is a 1:100 ratio for the minority class to the majority class. The invert of this ratio could be used with 1 for the majority class and 100 for the minority class.

for example:

```
# define model
weights = {0:1.0, 1:100.0}
model = DecisionTreeClassifier(class_weight=weights)
```

We might also define the same ratio using fractions and achieve the same results

```
# define model
weights = {0:0.01, 1:1.0}
model = DecisionTreeClassifier(class_weight=weights)
```

Or also can be defined by pre-loaded attributes: `python model = DecisionTreeClassifier(class_weight='balanced')

Detail about `class_weight` in `DecisionTreeClassifier` class

`**class_weightdict, list of dict or “balanced”, default=None**`

Weights associated with classes in the form `{class_label: weight}`. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y.

Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be `[{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}]` instead of `[{1:1}, {2:5}, {3:1}, {4:1}]`.

The “balanced” mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`

For multi-output, the weights of each column of y will be multiplied.

Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

```
model = DecisionTreeClassifier(class_weight='balanced')
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
scores = cross_val_score(model, X, y, scoring='roc_auc', cv=cv, n_jobs=-1)
```

```
print(f'The model ROC AUC mean is: {np.mean(scores):.3f}')
```

```
The model ROC AUC mean is: 0.749
```

Ok, very well, the performance just a little bit imporved, from `0.734` to `0.749`

Grid Search Weighted Decision Tree

Using a class weighting that is the inverse ratio of the training data is just a heuristic.

It is possible that better performance can be achieved with a different class weighting, and this too will depend on the choice of performance metric used to evaluate the model.

In this section, we will grid search a range of different class weightings for the weighted decision tree and discover which results in the best ROC AUC score.

We will try the following weightings for class 0 and 1:

- Class 0: 100, Class 1: 1
- Class 0: 10, Class 1: 1
- Class 0: 1, Class 1: 1
- Class 0: 1, Class 1: 10
- Class 0: 1, Class 1: 100

This can be defined as grid search parameter for the GridSearchCV class as follow:

```
# define grid
balance = [{0:100,1:1}, {0:10,1:1}, {0:1,1:1}, {0:1,1:10}, {0:1,1:100}]
param_grid = dict(class_weight=balance)
```

```
balance = [{0:100,1:1}, {0:10,1:1}, {0:1,1:1}, {0:1,1:10}, {0:1,1:100}]
param_grid = dict(class_weight=balance)
print(param_grid)
```

```
{'class_weight': [{0: 100, 1: 1}, {0: 10, 1: 1}, {0: 1, 1: 1}, {0: 1, 1: 10}, {0: 1, 1: 100}]}
```

```
from sklearn.model_selection import GridSearchCV

# define evaluation procedure
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)

# define model
```

```
model = DecisionTreeClassifier()

# define grid search
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=cv, scoring='roc_auc', verbose
# fit the GridSearch
grid_result = grid.fit(X, y)
```

```
Fitting 30 folds for each of 5 candidates, totalling 150 fits
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 12 concurrent workers.
[Parallel(n_jobs=-1)]: Done 150 out of 150 | elapsed:    0.4s finished
```

```
print(f'Best {grid_result.best_score_} using {grid_result.best_params_}')
```

```
Best 0.750925925925926 using {'class_weight': {0: 1, 1: 10}}
```

```
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

```
0.740556 (0.071391) with: {'class_weight': {0: 100, 1: 1}}
0.735539 (0.070241) with: {'class_weight': {0: 10, 1: 1}}
0.735707 (0.068985) with: {'class_weight': {0: 1, 1: 1}}
0.750926 (0.071732) with: {'class_weight': {0: 1, 1: 10}}
0.746212 (0.075781) with: {'class_weight': {0: 1, 1: 100}}
```

Very well, fine turning the `Class_weight` hyperparameter, we have `0.01` performance imporve

Further Reading

Paper

- An instance-weighting Method To Induce Cost-sensitive Tree, 2002

Books

- Learning from Imbalanced Datasets, 2018
- Imbalanced Learning: Foundations, Algorithms, and Applications, 2013.

APIs

- sklearn.utils.class_weight.compute_class_weight
- sklearn.tree.DecisionTreeClassifier
- sklearn.model_selection.GridSearchCV